

案例一

1.

解耦部门编码与主键：引入内部唯一标识作为部门主键，部门编码改为属性，支持编码变更而不影响关联数据。

建立部门历史映射表：记录部门编码变更前后的对应关系及生效时间，保留组织机构调整痕迹，支持业务数据追溯。

抽象部门关系规则：将友好/互斥关系从硬编码改为配置化，如关系表或规则引擎，便于动态调整，避免代码修改。

重构业务逻辑：将依赖具体编码的判断逻辑改为基于部门属性或关系表查询，减少对固定编码的耦合。

设计可扩展的组织结构模型：支持多版本组织机构树，允许按时间切片查询历史结构，适应未来调整而不需升级。

数据迁移与校验：编写数据迁移脚本，将业务数据中的旧编码映射到新部门内部 ID，并进行一致性校验，确保不丢失数据。

2.

软件工程方面：

采用增量与迭代开发，先搭建可扩展的核心模型，再逐步改造业务模块。

建立自动化测试，重点覆盖部门编码变更、关系判断、历史追溯场景。

进行代码重构与评审，消除硬编码，确保设计模式一致。

使用版本控制与持续集成，管理新旧代码并存与迁移过程。

项目管理方面：

制定质量计划，明确关键质量目标及验收标准。

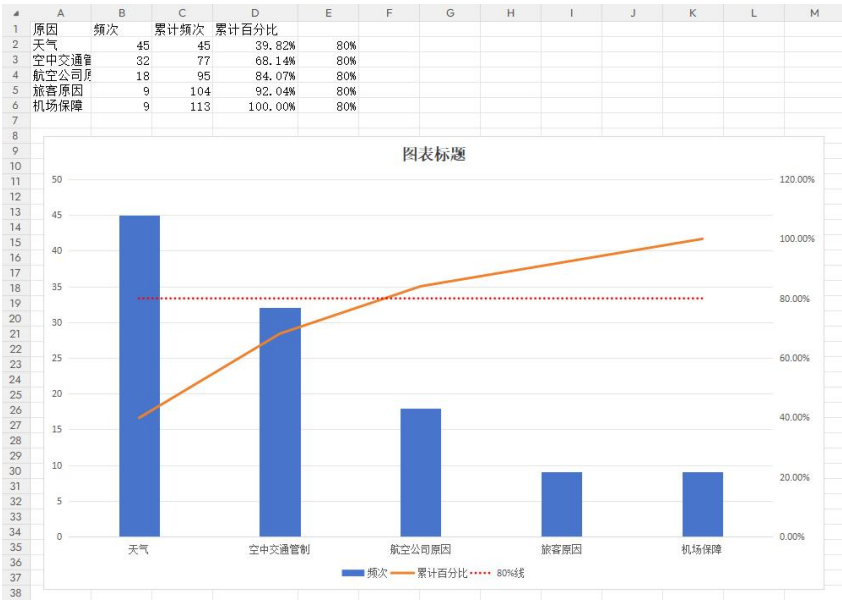
组织技术原型验证，降低设计风险。

加强配置管理与变更控制，对部门关系重新定义等变更进行影响分析。

设立里程碑评审，确保各阶段输出质量。

识别质量风险，制定应对措施。

案例二



案例三

1.

先解决最影响使用的问题，如系统崩溃、响应慢，并采取相应措施。

收集数据，记录所有问题的类型、频率、影响人数。

检查技术、数据、用户培训等问题，找根本原因。

用帕累托分析找到最重要的 20%原因优先处理。

建立性能测试、上线前压力测试、用户培训、SLA 监控等规范。

对比改进前后数据，把有效措施写入公司流程。

2.

明确问题及分类。

选择度量单位，如频次、影响时间、投诉数等。

收集数据，固定时间段内记录每类问题次数。

按度量值从大到小排序，计算累计百分比。

绘制帕累托图。

找出累计达 80%的关键少数原因。

优先解决这些关键原因。

改进后重新分析，验证效果。